

**Project Title: Exam Security
System Identity Verification
Seating Plan Violation
Logging**

Prepared by:

Berkay Aksoy

Number: 220706307

Hakkı Dökmeci

Number:2207060026

**Course: SE 342 01 Software Validation and
Testing Instructor: Emre Olca**

CONTENTS

1. SYSTEM ANALYSIS & REQUIREMENTS

2.Database Design

2.1 Database Schema

2.2 ERD Diagram

3.Sequence Diagram

4.Use-Case Diagram

5.Database Development

6.Page Designs

1.SYSTEM ANALYSIS & REQUIREMENTS

1.1. Functional Requirements (FR)

- **FR-01 Authentication:** The system shall allow users to log in using username/email and password.
- **FR-02 Role-Based Access:** The system shall authorize access based on roles: **Admin** (Exam Coordinator) and **Proctor**.
- **FR-03 Exam Management:** The system shall allow Admin to create, view, update, and delete exams (specifying date/time and room).
- **FR-04 Room & Seating Plan:**
 - The system shall allow Admin to create a seating plan for an exam room using seat codes (e.g., A1, A2...) or rows/columns.
 - The system shall allow Admin to assign students to seats for a specific exam.
- **FR-05 Student Roster:** The system shall allow Admin to import or manually add a student roster for an exam.
- **FR-06 Check-in Workflow:**
 - The system shall allow Proctor to check-in a student for an exam.
 - The system shall capture or upload a student photo during the check-in process.
- **FR-07 Identity Verification (ML/CV):**

- The system shall verify student identity by comparing the check-in photo with the registered ID photo.
- The system shall store the verification result (*Match / No Match / Not Processed*) for each check-in.
- **FR-08 Seating Compliance:**
 - The system shall validate whether the student is sitting in the assigned seat for that exam.
 - The system shall store the seat compliance result (*Correct / Incorrect*) for each check-in.
- **FR-09 Violation Logging:**
 - The system shall automatically create a violation record when identity verification fails.
 - The system shall automatically create a violation record when seating compliance fails.
 - The system shall allow Proctor to add notes and optional evidence image to a violation.
- **FR-10 Audit & Timestamp:** The system shall log each check-in attempt with a timestamp and the Proctor who performed it.
- **FR-11 Reporting:** The system shall allow Admin to generate basic reports, including:
 - List of check-ins
 - List of mismatches
 - List of seat violations
 - Summary counts per exam

1.2. Non-Functional Requirements (NFR)

- **NFR-01 Security (Authentication):** All users shall be authenticated before accessing any system feature.
- **NFR-02 Authorization (RBAC):** The system shall enforce role-based permissions for all endpoints and pages.
- **NFR-03 Performance:** The system shall respond to user actions within acceptable limits (e.g., X seconds) under normal load, particularly for check-in submission and report generation.
- **NFR-04 Reliability:** The system shall ensure data integrity and must not lose check-in or violation records once saved to the database.
- **NFR-05 Usability:** The check-in flow shall be designed to be completed in a minimal number of steps (e.g., Select Student → Upload/Capture Photo → Submit).
- **NFR-06 Maintainability:** The system architecture shall separate concerns (UI, Business Logic, Data Access, ML Wrapper/Service) to enable effective unit testing.

- **NFR-07 Logging:** The system shall log critical events (login, check-in, violation creation, exam creation) for auditing purposes.

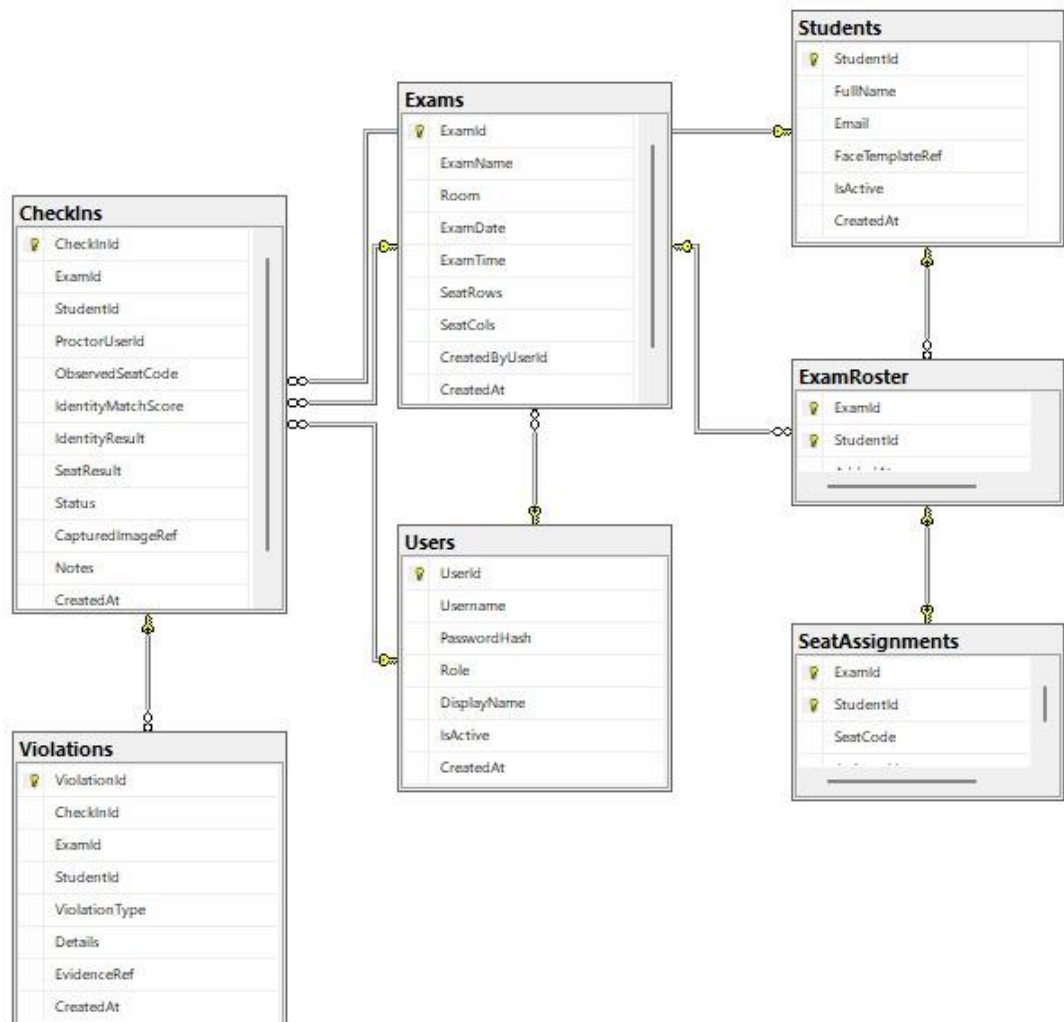
1.3. Business Rules & Validations (BR)

- **BR-01 Duplicate Check-in:**
 - A student cannot check in twice for the same exam.
 - If a check-in record already exists for the pair {StudentId, ExamId}, the system shall reject the request and return an appropriate error.
- **BR-02 Identity Verification Failure → Violation:**
 - If the identity verification result is **No Match** (or falls below a defined similarity threshold), the system shall automatically create a violation record with:
 - Reason: IDENTITY_MISMATCH
 - Timestamp: Current Time
 - Link: Related Check-in ID
- **BR-03 Seating Non-Compliance → Violation:**
 - If the seat reported during check-in does not match the assigned seat, the system shall automatically create a violation record with:
 - Reason: WRONG_SEAT
 - Timestamp: Current Time
 - Link: Related Check-in ID
- **BR-04 Exam Creation Permission:** Only the **Admin** role can create, update, or delete exam definitions (exam details, room, date/time, seating plan).
- **BR-05 Proctor Restrictions:**
 - A **Proctor** cannot edit exam definitions (room, date/time, seating plan, student-seat assignments).
 - A Proctor is authorized only to:
 - Perform check-ins.
 - View the exam roster.
 - Create/view violations (including adding notes/evidence).
- **BR-06 Required Data for Check-in:**
 - A check-in request must include the following fields: ExamId, StudentId, Photo (upload or capture), and Observed Seat Code.
 - If any required field is missing, the system shall reject the request.
- **BR-07 Student Must Exist in Exam Roster:**
 - A student can be checked in only if they are included in the roster for that specific exam.
 - Otherwise, the system must reject the request with the error: *"Student not registered for this exam."*
- **BR-08 Seating Plan Must Be Defined:**
 - Seating compliance can be evaluated only if a seating plan and student-seat assignments exist for the exam.

- If these are not defined, the system shall either block the check-in or mark seat compliance as NOT_AVAILABLE.
- **BR-09 Violation Integrity:**
 - Every violation must be linked to an ExamId and StudentId (and preferably a CheckInId).
 - Violation records must store a timestamp and reason; notes are optional.
- **BR-10 Evidence Optional:** The evidence image is optional. If provided, it must be a valid image format (e.g., .jpg, .png) and within defined file size limits.

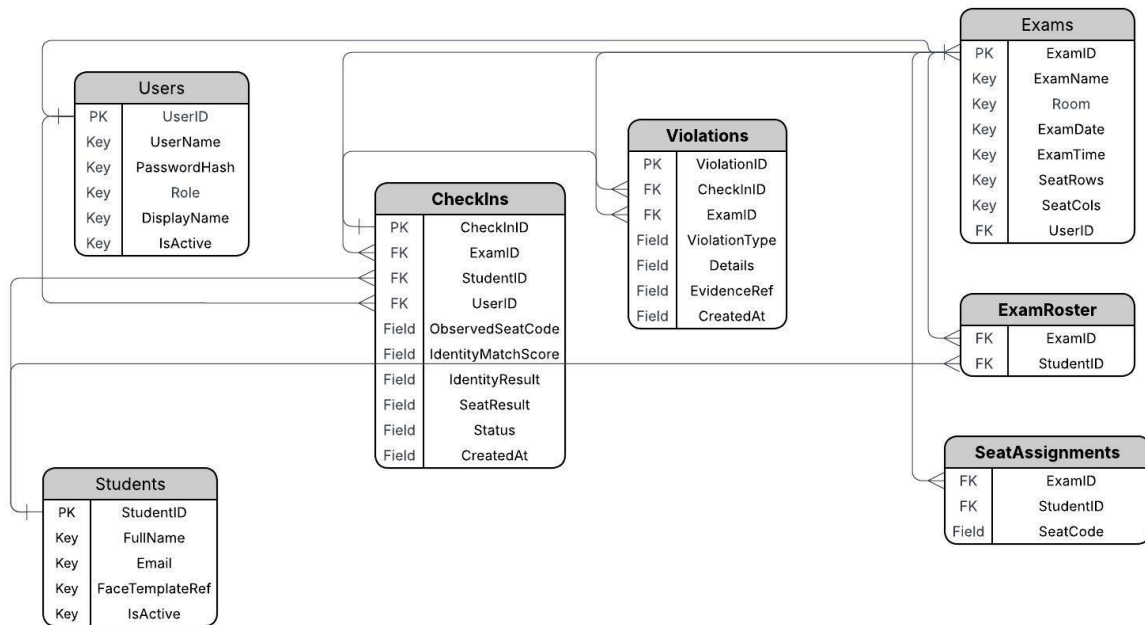
2.Database Design

2.1 Database Schema



This relational schema is designed to support the Exam Security System's core workflows. It connects **Users** (Admins/Proctors) and **Students** to specific **Exams** through roster and seating assignment tables. The central **CheckIns** table records real-time identity verification results and seating compliance, while the **Violations** table automatically logs any detected security anomalies or manual incident reports.

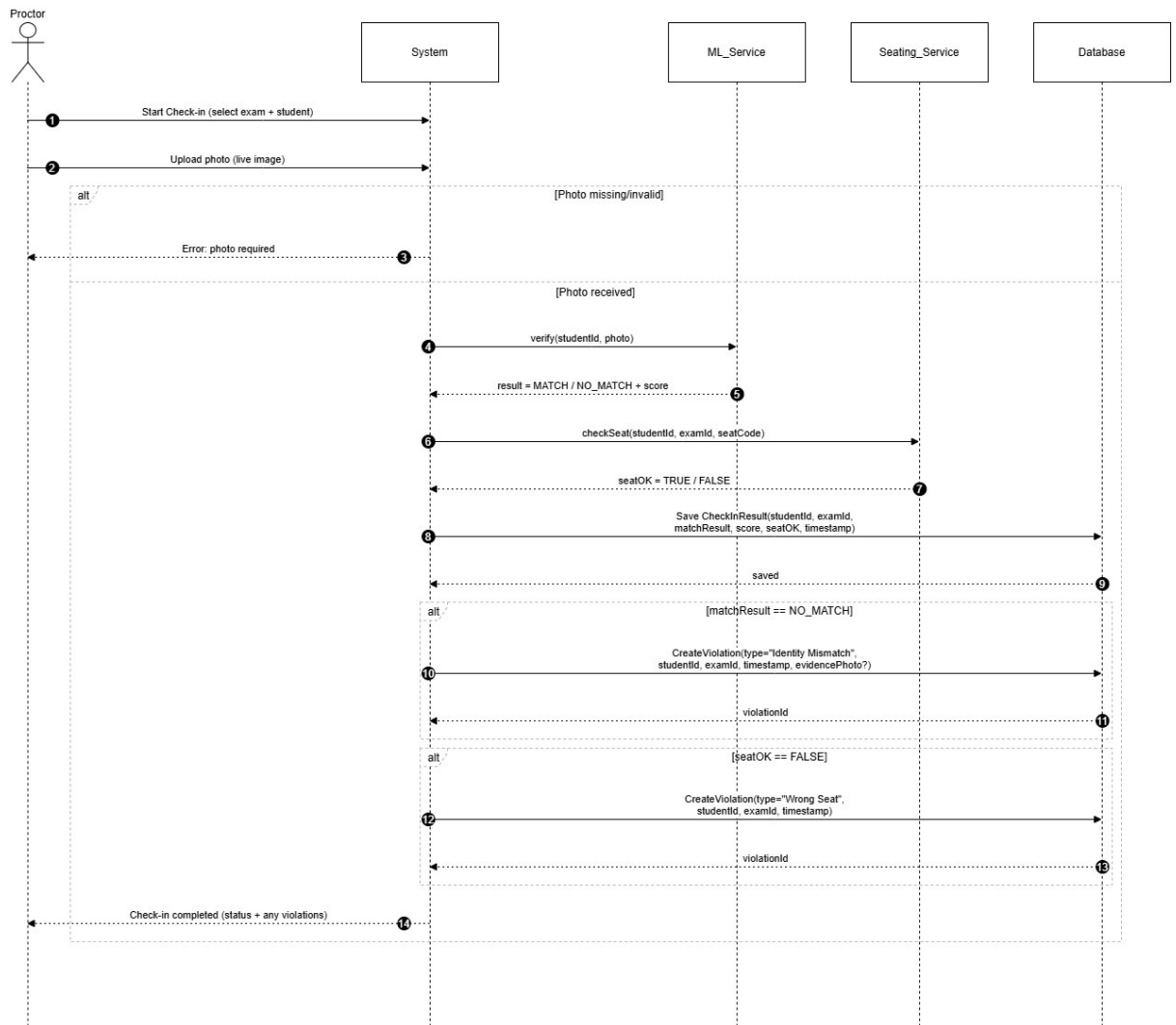
2.2 ERD Diagram



This diagram illustrates the relational database schema designed to support the core workflows of the Exam Security System. The model is organized into logical segments:

- **User & Exam Management:** The Users table manages role-based access for Admins and Proctors. The Exams table defines exam metadata, including room layout dimensions (SeatRows, SeatCols).
- **Student Association:** The Students table stores identity data, including facial recognition references (FaceTemplateRef). The ExamRoster and SeatAssignments linking tables manage the Many-to-Many relationships, ensuring students are registered for exams and assigned specific seats.
- **Check-in Workflow:** The CheckIns table acts as the primary transactional entity, logging the timestamp, the acting Proctor (ProctorUserId), the ML verification result (IdentityMatchScore), and the seating compliance status.
- **Incident Logging:** The Violations table is directly linked to CheckIns to capture security anomalies, storing the violation type, details, and optional evidence references (EvidenceRef).

3. Sequence Diagram

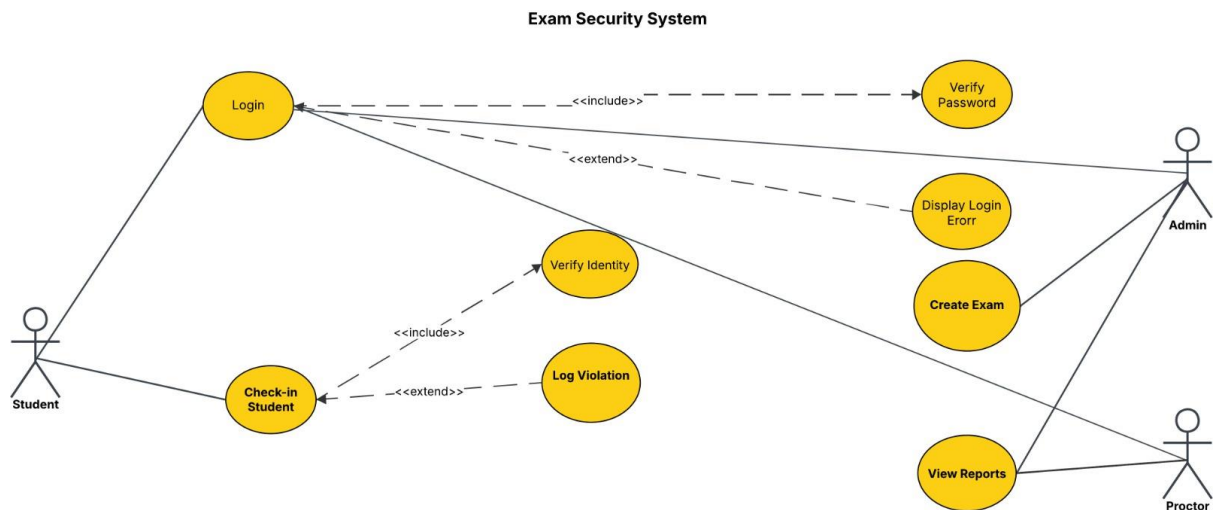


This diagram illustrates the interaction between the Proctor and the System components during the student check-in process. It details the orchestration of external calls to the ML_Service for identity verification and the Seating_Service for compliance checks.

The diagram explicitly defines conditional flows using alt fragments:

- **Validation:** Handles missing or invalid photo inputs.
- **Violation Logic:** Automatically triggers a CreateViolation call to the Database if the identity match fails (NO_MATCH) or if the seating assignment is incorrect (seatOK == FALSE).

4. Use Case Diagram



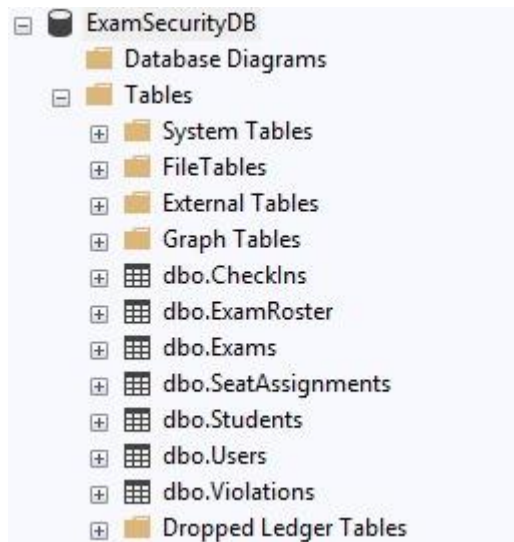
This diagram depicts the functional scope of the system and the interactions between the primary actors: **Student**, **Admin**, and **Proctor**.

Key architectural relationships are defined as follows:

- **Authentication:** The "Login" case includes a mandatory "Verify Password" step and conditionally extends to "Display Login Error" if authentication fails.
- **Check-in Workflow:** The "Check-in Student" use case has a mandatory dependency (<<include>>) on "Verify Identity," ensuring ML verification runs for every attempt. Conversely, "Log Violation" is modeled as an extension (<<extend>>), triggering only when specific security rules are breached.

5.Database Development

The database layer of the system was implemented based on the previously designed ERD (Entity Relationship Diagram). All entities, attributes, relationships, and constraints defined in the ERD were successfully translated into a fully normalized relational database structure using Microsoft SQL Server. The final schema includes seven core tables—**Users**, **Students**, **Exams**, **ExamRoster**, **SeatAssignments**, **CheckIns**, and **Violations**—ensuring data integrity, scalability, and support for the system's functional requirements.



This screenshot demonstrates the successful deployment of the relational schema within the Microsoft SQL Server environment. The database, named ExamSecurityDB, contains the seven core tables defined in the system design phase:

- dbo.Users, dbo.Students, dbo.Exams (Core Entities)
- dbo.ExamRoster, dbo.SeatAssignments (Linking Tables)
- dbo.CheckIns, dbo.Violations (Operational Tables)

This visual evidence confirms that the data persistence layer is fully initialized and structurally aligned with the Entity Relationship Diagram (ERD).

5.1 CheckIns Table

	Column Name	Data Type	Allow Nulls
▶	CheckInId	int	☐
	ExamId	int	☐
	StudentId	int	☐
	ProctorUserId	int	☐
	ObservedSeatCode	nvarchar(10)	☐
	IdentityMatchScore	int	☒
	IdentityResult	nvarchar(20)	☐
	SeatResult	nvarchar(20)	☐
	Status	nvarchar(20)	☐
	CapturedImageRef	nvarchar(300)	☒
	Notes	nvarchar(500)	☒
	CreatedAt	datetime2(7)	☐
			☐

The CheckIns table functions as the central transactional entity for the system's runtime operations. It persists the outcome of the proctor's validation workflow for every student entry. Key attributes include:

- **Validation Logic Storage:** It stores the quantitative output from the ML service (IdentityMatchScore) and the qualitative status of the seating compliance check (SeatResult).
- **Audit Trail:** By recording the ProctorUserId and CreatedAt timestamp, the system ensures full accountability for every verification decision.
- **Operational Status:** The Status column tracks the workflow state (e.g., 'Verified', 'Violation Detected'), while CapturedImageRef links to the physical evidence file stored in the system.

5.2 ExamRoster Table

	Column Name	Data Type	Allow Nulls
▶	ExamId	int	☐
▶	StudentId	int	☐
	AddedAt	datetime2(7)	☐
			☐

This table functions as a junction entity designed to resolve the Many-to-Many relationship between Exams and Students. It serves as the definitive registry (whitelist) for exam eligibility.

- **Relationship Management:** It links specific students to specific exam sessions via the ExamId and StudentId foreign keys.

- **Data Integrity:** The combination of ExamId and StudentId forms a **Composite Primary Key**, physically preventing duplicate registrations for the same student in the same exam.
- **Audit:** The AddedAt column tracks exactly when a student was added to the roster, supporting administrative auditing.

	Column Name	Data Type	Allow Nulls
	ExamId	int	☐
	ExamName	nvarchar(200)	☐
	Room	nvarchar(50)	☐
	ExamDate	date	☐
	ExamTime	time(0)	☐
	SeatRows	int	☐
	SeatCols	int	☐
	CreatedByUserId	int	☐
	CreatedAt	datetime2(7)	☐
			☐

This table functions as a junction entity designed to resolve the Many-to-Many relationship between Exams and Students. It serves as the definitive registry (whitelist) for exam eligibility.

- **Relationship Management:** It links specific students to specific exam sessions via the ExamId and StudentId foreign keys.
- **Data Integrity:** The combination of ExamId and StudentId forms a **Composite Primary Key**, physically preventing duplicate registrations for the same student in the same exam.
- **Audit:** The AddedAt column tracks exactly when a student was added to the roster, supporting administrative auditing.

5.3 SeatAssignments Table

	Column Name	Data Type	Allow Nulls
🔑	ExamId	int	☐
🔑	StudentId	int	☐
	SeatCode	nvarchar(10)	☐
	AssignedAt	datetime2(7)	☐
			☐

The SeatAssignments table defines the mandatory seating plan for each exam session. It maps a specific StudentId within a specific ExamId to a designated SeatCode (e.g., "A1", "B5").

- **Compliance Logic:** This table serves as the "truth source" for the Seating Compliance business rule. During the check-in process, the system queries this table to validate if the ObservedSeatCode matches the assigned seat.
- **Structure:** It uses a composite primary key (ExamId + StudentId) to ensure that a student is assigned to exactly one seat per exam.

5.4 Students Table

	Column Name	Data Type	Allow Nulls
🔑	StudentId	int	☐
	FullName	nvarchar(150)	☐
	Email	nvarchar(150)	☒
	FaceTemplateRef	nvarchar(300)	☒
	IsActive	bit	☐
	CreatedAt	datetime2(7)	☐
			☐

The Students table serves as the master data repository for student identities within the system. It plays a pivotal role in the biometric verification workflow:

- **Biometric Reference:** The FaceTemplateRef column stores the file path or embedding reference to the student's registered ID photo. This data is retrieved by the ML Service during the check-in process to compare against the live camera feed.
- **Identity Management:** It creates a unique StudentId for every individual, which is then referenced by the ExamRoster and CheckIns tables to track attendance and eligibility.

5.5 Users Table

	Column Name	Data Type	Allow Nulls
🔑	UserId	int	☐
	Username	nvarchar(100)	☐
	PasswordHash	nvarchar(200)	☐
	Role	nvarchar(20)	☐
	DisplayName	nvarchar(150)	☐
	IsActive	bit	☐
	CreatedAt	datetime2(7)	☐
			☐

The Users table implements the **Authentication and Authorization** layer of the system. It stores secure credentials for all system operators.

- **RBAC Enforcement:** The Role column acts as the discriminator for Role-Based Access Control, strictly categorizing users as 'Admin' or 'Proctor'.
- **Security:** Instead of plain text, passwords are stored in the PasswordHash column. The IsActive flag allows administrators to instantly revoke access without deleting the user record.

5.6 Violations Table

	Column Name	Data Type	Allow Nulls
🔑	ViolationId	int	☐
	CheckInId	int	☐
	ExamId	int	☐
	StudentId	int	☐
	ViolationType	nvarchar(50)	☐
	Details	nvarchar(500)	☒
	EvidenceRef	nvarchar(300)	☒
	CreatedAt	datetime2(7)	☐
			☐

The Violations table serves as the persistent log for all security incidents detected by the system. It is tightly coupled with the CheckIns table to provide context for failed verifications.

- **Incident Logging:** It captures specific breach types (ViolationType), such as "Identity Mismatch" or "Seating Violation," ensuring that every anomaly is categorized.

- **Forensic Data:** The table stores references to digital evidence (EvidenceRef) and allows proctors to append explanatory notes (Details), creating a comprehensive audit trail for disciplinary actions.

Brkays.ExamS... dbo.CheckIns												
	CheckInId	ExamId	StudentId	ProctorUserId	ObservedSe...	IdentityMatc...	IdentityResult	SeatResult	Status	CapturedIma...	Notes	CreatedAt
▶	1	1	202001	2	A1	98	MATCH	OK	CHECKED_IN	NULL	NULL	2026-01-09 0...
	2	1	202002	2	B3	95	MATCH	NOT_OK	VIOLATION	NULL	Observed sea...	2026-01-09 0...
	3	1	202003	2	A3	30	NO_MATCH	OK	VIOLATION	NULL	Face verificati...	2026-01-09 0...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Brkays.ExamS...o.ExamRoster			
	ExamId	StudentId	AddedAt
▶	1	202001	2026-01-09 0...
	1	202002	2026-01-09 0...
	1	202003	2026-01-09 0...
	1	202004	2026-01-09 0...
	1	202005	2026-01-09 0...
	3	202001	2026-01-09 1...
	3	202002	2026-01-09 1...
	3	202003	2026-01-09 1...
*	NULL	NULL	NULL

Brkays.ExamS...B - dbo.Exams									
	ExamId	ExamName	Room	ExamDate	ExamTime	SeatRows	SeatCols	CreatedByUs...	CreatedAt
▶	1	CS305 Final Si...	A-101	2026-01-15	10:00:00	4	6	1	2026-01-09 0...
	2	SE Final	B-201	2026-01-20	10:00:00	5	6	1	2026-01-09 1...
	3	SE Final	B-201	2026-01-20	10:00:00	5	6	1	2026-01-09 1...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Brkays.ExamS...tAssignments				
	ExamId	StudentId	SeatCode	AssignedAt
▶	1	202001	A1	2026-01-09 0...
	1	202002	A2	2026-01-09 0...
	1	202003	A3	2026-01-09 0...
	1	202004	B1	2026-01-09 0...
	1	202005	B2	2026-01-09 0...
*	NULL	NULL	NULL	NULL

Brkays.ExamS...tAssignments				
	ExamId	StudentId	SeatCode	AssignedAt
▶	1	202001	A1	2026-01-09 0...
	1	202002	A2	2026-01-09 0...
	1	202003	A3	2026-01-09 0...
	1	202004	B1	2026-01-09 0...
	1	202005	B2	2026-01-09 0...
*	NULL	NULL	NULL	NULL

Brkays.ExamS... dbo.Students						
	StudentId	FullName	Email	FaceTemplat...	IsActive	CreatedAt
▶	202001	Ahmet Yilmaz	ahmet.yilmaz...	NULL	True	2026-01-09 0...
	202002	Ayşe Demir	ayse.demir@u...	NULL	True	2026-01-09 0...
	202003	Mehmet Kaya	mehmet.kaya...	NULL	True	2026-01-09 0...
	202004	Zeynep Arslan	zeynep.arslan...	NULL	True	2026-01-09 0...
	202005	Can Özkan	can.ozkan@u...	NULL	False	2026-01-09 0...
*	NULL	NULL	NULL	NULL	NULL	NULL

Brkays.ExamS...B - dbo.Users							
	UserId	Username	PasswordHash	Role	DisplayName	IsActive	CreatedAt
▶	1	admin	Admin123	Admin	Exam Coordin...	True	2026-01-09 0...
	2	proctor	1234	Proctor	Proctor User	True	2026-01-09 0...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Brkays.ExamSe...bo.Violations								
	ViolationId	CheckId	ExamId	StudentId	ViolationType	Details	EvidenceRef	CreatedAt
▶	1	2	1	202002	WRONG_SEAT	Assigned: A2, ...	NULL	2026-01-09 0...
	2	3	1	202003	IDENTITY_MIS...	Match score t...	NULL	2026-01-09 0...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6. Page Designs

6.1 Login Interface Design

The Login interface serves as the secure entry point for the Exam Security System. It is designed with a minimalist approach to minimize user error and enhance security.

- **Authentication:** Accepts Username and Password credentials to validate identity against the Users database table.
- **Role-Based Redirection:** Upon successful login, the system automatically detects the user's role (Admin or Proctor) and redirects them to their respective dashboards.
- **Feedback Mechanism:** Provides real-time visual feedback for invalid credentials or connection errors to ensure a smooth user experience.

To ensure a responsive user experience, the interface includes an immediate validation mechanism. If authentication fails due to incorrect credentials, the system triggers a visual alert modal to explicitly inform the user, preventing ambiguity and guiding them to retry.

6.2 Exam Configuration Interface

ExamSecurity | Final Project

Exam SetupReportsLogout

Sınav Oluşturma & Yönetim

1. Sınav Detayları (Exam Details)

Sınav Adı

CS305 Final Sınavı

Sınav Salonu (Room)

A-101

Tarih

15.01.2026

Saat

10:00

This interface represents the administrative module for scheduling and configuring exam sessions. It is restricted to the **Admin** role to ensure the integrity of exam schedules.

- **Data Binding:** The form fields (Exam Name, Room, Date, Time) are directly mapped to the `dbo.Exams` database table, ensuring that all inputs are structured correctly for storage.
- **Usability:** The design utilizes standard input controls, including date and time pickers, to minimize formatting errors during data entry.
- **Context:** As shown in the screenshot, the admin is currently configuring the "CS305 Final Exam" for Room "A-101".

6.3 Student Roster Management Interface

2. Öğrenci Listesi (Roster)

Excel (.xlsx) veya CSV formatında öğrenci listesini yükleyin.

Dosya Seç

Dosya seçilmedi

Listeyi Yükle

STUDENTID	FULLNAME	EMAIL	ACTIVE	ASSIGNED SEAT
202001	Ahmet Yılmaz	ahmet.yilmaz@univ.edu.tr	True	A1
202002	Ayşe Demir	ayse.demir@univ.edu.tr	True	A2
202003	Mehmet Kaya	mehmet.kaya@univ.edu.tr	True	A3
202004	Zeynep Arslan	zeynep.arslan@univ.edu.tr	True	B1
202005	Can Özkan	can.ozkan@univ.edu.tr	False	B2

This section illustrates the interface used for populating and managing the list of eligible students for a specific exam session.

- **Bulk Data Integration:** The design features a file upload mechanism ("Dosya Seç" / "Listeyi Yükle"), allowing administrators to import student records in bulk via Excel (.xlsx) or CSV formats, significantly reducing manual data entry effort.
- **Database Synchronization:** The data grid dynamically retrieves and displays records from the `dbo.Students` and `dbo.ExamRoster` tables. It provides a consolidated view of critical attributes such as `StudentId`, `Email`, and `AssignedSeat`.
- **Status Visualization:** To enhance readability, the 'Active' column employs color-coded badges (Green for True, Red for False), giving immediate visual feedback on a student's eligibility status within the system.

6.4 Dynamic Seating Plan & Status Monitor

3. Oturma Düzeni (Seating Plan)

Satır Sayısı: 4 Sütun Sayısı: 6 [Önizle](#)

A1	A2	A3	A4	A5	A6
B1	B2	B3	B4	B5	B6
C1	C2	C3	C4	C5	C6
D1	D2	D3	D4	D5	D6

[Sınavı Kaydet](#)

Assigned Checked-in Violation (SeatAssignments + Checkins tablosuna göre)

This interface provides a spatial visualization of the exam environment, allowing administrators to configure the physical layout and monitor student status in real-time.

- **Grid Generation Algorithm:** The system dynamically renders a seat grid ($N \times M$) based on the user-defined `SeatRows` and `SeatCols` inputs, which are stored in the `dbo.Exams` table.
- **Real-time Status Visualization:** The interface acts as a dashboard by retrieving data from the `CheckIns` and `SeatAssignments` tables. It utilizes a color-coded state management system:
 - **Green:** Successful Check-in (Verified).
 - **Blue:** Seat Assigned (Pending).
 - **Red:** Violation Detected (e.g., Wrong Seat).

- **Operational Control:** This visual feedback loop enables proctors and admins to identify and address security anomalies immediately without parsing through raw text logs.

LOGIN AS A PROCTOR

The screenshot shows the 'Sisteme Giriş' (System Login) page. The header includes the 'ExamSecurity | Final Project' logo and navigation links for 'Check-in' and 'Logout'. The main content area features a login form with the following fields: 'Kullanıcı Adı' (Username) with the value 'proctor', 'Şifre' (Password) with masked characters '****', and 'Giriş Rolü (Role)' (Login Role) with a dropdown menu set to 'Proctor (Gözetmen)'. A blue button labeled 'Güvenli Giriş Yap →' (Secure Login) is positioned below the form. At the bottom, a note states 'Demo Users: admin/Admin123, proctor/1234'.

6.5 Proctor Check-in Station

The screenshot displays the 'Gözetmen Paneli (Check-in Station)' interface. The header shows 'ExamSecurity | Final Project' and navigation links for 'Check-in' and 'Logout'. The main content area is divided into two sections. On the left, the 'Öğrenci Doğrulama' (Student Verification) section includes a dropdown for 'Öğrenci Seçiniz (Listeden)' (Select Student from List) with the value '202001 - Ahmet Yılmaz (Seat: A1, Active)', a text input for 'Gözlemlenen Sıra No' (Observed Seat No) with the value 'A1', and a video feed of a student. Below the video feed are buttons for 'Çek' (Take) and 'Yükle' (Upload). On the right, a green box displays a large green checkmark and the text 'Giriş Onaylandı' (Login Approved) with a 'Timestamp: 16:43:27'. Below this, two status messages are shown: '✓ Kimlik (ML): 98% Eşleşme (MATCH)' and '✓ Oturma Düzeni: Doğru (Assigned: A1, Observed: A1)'. A 'Launchpad' button is visible in the bottom left corner.

This interface is the primary operational dashboard for proctors, designed to execute the multi-factor authentication workflow.

- **Workflow Integration:** The proctor selects a student from the dynamic roster (retrieved from `dbo.ExamRoster`) and inputs the observed seat code.

- **Biometric Analysis:** The "Camera Feed" component captures the student's live image, which is processed by the ML service to generate an IdentityMatchScore. In this specific instance, the system returned a **98% Match**, validating the student's identity against the reference template in dbo.Students.
- **Decision Support:** The result panel provides immediate, color-coded feedback. The green "Giriş Onaylandı" (Entry Approved) status confirms that both the Identity Check and Seating Compliance (Seat A1) rules have been satisfied, and a transaction record has been committed to the dbo.CheckIns table.

ExamSecurity | Final Project Check-in Logout

Gözetmen Paneli (Check-in Station)

Öğrenci Doğrulama

Öğrenci Seçiniz (Listeden)

202003 - Mehmet Kaya (Seat: A3, Active)

Gözetimlenen Sıra No

A3

İpucu: Student'in assigned seat'ini yanlış girersen **WRONG_SEAT** violation çıkar.

İhlal Tespit Edildi!

Reason: IDENTITY_MISMATCH (Match score: 30% (NO_MATCH))

Gözetmen Notları:

Face verification failed.

Kanıt Görseli (Opsiyonel):

Dosya seçilmedi

Not: Violation kaydı oluşturunca Reports ekranı da güncellenir.

Security Incident & Violation Logging Interface This figure demonstrates the system's response to a failed biometric verification attempt, illustrating the enforcement of the "Identity Verification" business rule.

- **Anomaly Detection:** In this scenario, while the student ("Mehmet Kaya") is in the correct seat (A3), the Machine Learning service returned a **Match Score of 30%**, which is below the acceptance threshold. Consequently, the system flagged the transaction as IDENTITY_MISMATCH.
- **Visual Alert System:** The interface immediately shifts to a high-contrast "Red Alert" state to warn the proctor. This prevents accidental entry approval.
- **Incident Persistence:** The form automatically prepares a violation record. The proctor's notes ("Face verification failed") and the incident type are structured to be inserted directly into the dbo.Violations table, creating an immutable audit trail.